

dev76 — CUDA C++ MultiGrid 粗层求解性能优化（多 block 归约 + fused 阈值）

opencode analy

2026 年 8 月 16 日

摘要

本报告分析 PyQCU 仓库 dev76 里程碑：多线程版（一线程一卡）CUDA C++ MultiGrid 求解器的粗层求解性能优化。核心改进在 `cpp/cuda/qcu/include/ lattice_clover_multigrid.h`：粗层点积多 block 归约（`coarse_dot_kernel_multi + coarse_dot_reduce_kernel`）；大最粗层（ $\geq 64K$ 元素）从 fused cooperative kernel 回退普通多 block 迭代路径。三视角概览：主体结构为 Python 编排 + C++ CUDA 内核双层级；各部分关系为粗层 BiStabCG 迭代由 14 个小内核 + host sync 构成，点积归约是其主导开销；项目思路为消除单 block 串行归约的并行度瓶颈，用实测数据驱动（coarse solve 533ms 定位主导项）。核心结论：16x16x16x32 3L 的粗层求解 533ms $\rightarrow$ 42ms（约 10 倍）；P100 $\times$ 2 8x8x8x16 2L 加速比 1.83 $\rightarrow$ 2.01-2.34；V100 16x16x16x32 3L 从 <1 提升至约 1.0-1.2。

目录

1 仓库概览	2
2 主体结构	2
3 各部分关系	2
4 项目思路	3
4.1 设计意图	3
4.2 演进脉络	3
4.3 典型工作流	3
5 主题分析	3
5.1 瓶颈定位	3
5.2 多 block 归约	4
5.3 实测结果	4
6 关键代码/文档片段	4
7 参考源清单	6

1 仓库概览

PyQCU 为格点 QCD (Lattice QCD) 数值库: Wilson/Clover Dirac 算子、BiStabCG 与多重网格求解器、stout smearing、规范场生成, 经 MPI 分布于 4D 进程网格。本里程碑聚焦 C++ CUDA 后端的 MultiGrid 求解器粗层路径。

指标	实测值
本里程碑改动	lattice_clover_multigrid.h (+13/-5 工作树增量)
涉及内核	coarse_dot_kernel / coarse_dot_kernel_multi / coarse_dot_reduce_kernel / multigrid_coarse_solve_cg
测试套件	logs/dev76/ (main.py 子命令 + v<ts>/ 版本目录 + h5py)

表 1: dev76 里程碑实测统计 (数据来源: git/find 实测)

2 主体结构

MultiGrid 求解器 (lattice_clover_multigrid.h) 的分层结构:

层次	代码位置	职责
入口层	logs/dev76/main.py	测试编排 (verify/clean/bench/sweep/check)
桥接层	pyqcu/cuda/_multi_gpu.py	MultiGpuMultigrid 多线程多卡驱动
C++ 求解层	lattice_clover_multigrid.h	V-cycle、BiStabCG 迭代、fused 粗解
内核层	multigrid.cu	33-tensor 粗层 dslash、fused 求解内核

表 2: 主体结构分层 (数据来源: 全库索引实测)

本次改动集中在 C++ 求解层: 新增多 block 归约 kernel 与二次归约 kernel (见第 6 节代码片段), 并修改 dot_coarse、bistabcg_iter_coarse、v_cycle 三处调用点。

3 各部分关系

要点

依赖主链: logs/dev76/main.py → pyqcu/cuda/_multi_gpu.py (MultiGpuMultigrid) → qcu.pyx (Cython) → libqcu.so → lattice_clover_multigrid.h (V-cycle) → bistabcg_iter_coarse → coarse_dot_kernel_multi + coarse_dot_reduce_kernel

粗层 BiStabCG 单次迭代 (bistabcg_iter_coarse) 的构成:

- 5 次点积 (rho / rtv / ts / tt / norm) ——本次优化的对象；
- 2 次 33-tensor 粗层 dslash (coarse_dslash_op) ——并行度充足；
- 1 次 D2H 镜像 + host sync (收敛检查) —— 170 μ s 固定开销。

版本演进：test14 (粗算子构建加速) → dev76 (粗层求解加速，本次)。

4 项目思路

4.1 设计意图

- **消除串行归约瓶颈**：单 block 256 线程归约 196608 元素时每线程串行累加 768 次，GPU 并行度完全浪费——grid-stride 多 block + 二次归约恢复并行；
- **按规模选择路径**：小向量 (<64K) 单 block 启动开销更低；大向量多 block 并行度高；最粗层按规模在 fused cooperative 与普通迭代间选择；
- **测量驱动**：每个优化方向先 profile (PROF_SECTIONS) 定位主导项，再实施最小改动。

4.2 演进脉络

dev76 起点是 test14 的已知短板：16x16x16x32 3L 的中间粗层 (lev=1) 求解占 coarse solve 533ms。先后尝试中间层全 fused (并行度不足，回退)、CHECK_INTERVAL 延迟检查 (破坏递归校正，回退)，最终锁定多 block 归约 (保留)。

4.3 典型 workflow

build.sh (重建 libqcu.so) → measure_multi (单配置验证) → PROF_SECTIONS 分析 → bench/sweep (完整回归) → 版本目录归档。

项目思路

项目思路核心：粗层 BiStabCG 的瓶颈是点积归约的并行度，而非 dslash 计算本身；按向量规模自适应选择归约/求解路径；任何优化以实测收益为准。

5 主题分析

5.1 瓶颈定位

PROF_SECTIONS 实测 (16x16x16x32 3L V100)：coarse_solve=533.7ms (5 次 V-cycle) vs fine_iter=248.8ms——粗层求解占主导。其中粗层 dslash 仅 0.4ms (prof_coarse_dslash_ms)，说明瓶颈在点积归约与内核启动/同步开销，而非算子本身。

5.2 多 block 归约

- `coarse_dot_kernel_multi`: grid-stride 归约, 每 block 产出 `partials[bid]`;
- `coarse_dot_reduce_kernel`: 单 block 对 ≤ 256 个 `partials` 二次归约;
- 阈值: `vec_sz  $\geq$  64K` 用多 block ($16 \times 16 \times 16 \times 32$ lv1 = 196608 元素), 否则保留单 block (启动开销更低)。

5.3 实测结果

配置	test14	dev76
16x16x16x32 3L coarse_solve	533.7ms	42ms (约 10 倍)
P100x2 8x8x8x16 2L speedup	1.83	2.01-2.34
P100x2 8x8x8x16 3L speedup	2.05	2.14-2.16
V100 16x16x16x16 3L speedup	1.19	1.12-1.33
V100 16x16x16x32 3L speedup	0.75	0.95-1.20
sweep 通过率 (gate=1.5)	12/16	14/16

表 3: test14 vs dev76 关键指标 (数据来源: 本次本机实测)

verify 全 PASS (一致性 `rel=0`)。2L 大格子 ($8 \times 16 \times 16 \times 16$ 2L、 $16 \times 16 \times 16 \times 16$ 2L) 仍 < 1 : 层数选择的固有特性 (2L 粗层无递归校正、条件数差), 3L 为正确配置。

6 关键代码/文档片段

多 block 归约 kernel (grid-stride + 二次归约):

```

1   if (idx < s) sdata[idx] += sdata[idx + s];
2   __syncthreads();
3   }
4   if (idx == 0) out[0] = sdata[0];
5   }
6
7   // Multi-block grid-stride reduction: partials[bid] = block-local <a,b> slice.
8   template <typename T, int NT>
9   __global__ void coarse_dot_kernel_multi(const LatticeComplex<T> *a,
10                                          const LatticeComplex<T> *b, int n,
11                                          LatticeComplex<T> *partials) {
12   __shared__ LatticeComplex<T> sdata[NT];
13   int idx = threadIdx.x, bid = blockIdx.x, nblk = gridDim.x;
14   int stride = nblk * NT;
15   LatticeComplex<T> sum(0, 0);
16   for (int i = bid * NT + idx; i < n; i += stride) sum += a[i].conj() * b[i];
17   sdata[idx] = sum;
18   __syncthreads();
19   for (int s = NT / 2; s > 0; s >>= 1) {
20       if (idx < s) sdata[idx] += sdata[idx + s];

```

```

21     __syncthreads();
22 }
23 if (idx == 0) partials[bid] = sdata[0];
24 }
25
26 // Second reduction over nblk partials (1 block).
27 template <typename T, int NT>
28 __global__ void coarse_dot_reduce_kernel(const LatticeComplex<T> *partials,
29                                         int nblk, LatticeComplex<T> *out) {
30     __shared__ LatticeComplex<T> sdata[NT];
31     int idx = threadIdx.x;
32     LatticeComplex<T> sum(0, 0);
33     for (int i = idx; i < nblk; i += NT) sum += partials[i];
34     sdata[idx] = sum;
35     __syncthreads();
36     for (int s = NT / 2; s > 0; s >>= 1) {
37         if (idx < s) sdata[idx] += sdata[idx + s];
38     }
39 }

```

bistabcg_iter_coarse 中按规模选择归约路径:

```

1  const LatticeComplex<T>* rt = static_cast<const LatticeComplex<T>*>(st.r_tilde);
2  const LatticeComplex<T>* r = static_cast<const LatticeComplex<T>*>(st.r);
3  auto tc0=std::chrono::high_resolution_clock::now();
4  // 2026-08-15 dev76: multi-block dot for large coarse vectors
5  // (single-block serial reduction dominated the coarse solve on
6  // 16x16x16x32 lv1 = 196608 elements).
7  const int n = (int)st.vec_sz;
8  const bool mb = (n >= 65536);
9  int nblk = (n + 255) / 256; if (nblk > 256) nblk = 256;
10 auto cdot = [&](const LatticeComplex<T>* a, const LatticeComplex<T>* b,
11               int slot) {
12     if (mb) {
13         coarse_dot_kernel_multi<T, 256><<<nblk, 256, 0, S>>>(
14             a, b, n, static_cast<LatticeComplex<T>*>(coarse_partials));
15         coarse_dot_reduce_kernel<T, 256><<<1, 256, 0, S>>>(
16             static_cast<const LatticeComplex<T>*>(coarse_partials), nblk, &dv[slot]);
17     } else {
18         coarse_dot_kernel<T, 256><<<1, 256, 0, S>>>(a, b, n, &dv[slot]);
19     }
20 };
21
22 // 1. rho = <r_tilde, r>
23 cdot(rt, r, _rho_);

```

fused 最粗层阈值 (vec_sz < 64K 才 fused):

```

1  // WSL2/V100 box (~14 tiny kernel launches + 1 host sync per iteration;
2  // the wide dslash is only ~2 us). multigrid_coarse_solve_cg() fuses the
3  // ENTIRE BiStabCG solve into ONE cooperative kernel with grid.sync()
4  // barriers — no host syncs inside. Valid at the coarsest level only.
5  //
6  // 2026-08-15 dev76: for LARGE coarsest levels (vec_sz >= 64K, e.g.
7  // 16x16x16x16 2L coarse [8,8,8,4] = 98304 elements) the fused kernel
8  // is NOT faster: its per-iteration grid.sync() barriers * many

```

```
9 // iterations cost more than the ordinary multi-block path (measured
10 // 92 ms vs 7 ms per V-cycle). Fall back to the ordinary iterative
11 // path below (which now uses the multi-block reduction kernels).
12 // =====
13 if (lev == num_levels - 1 && st.vec_sz < 65536) {
14     // Fused single-launch coarse solve. In the redundant-global multi-rank
15     // model every rank holds the full coarse grid and the fused kernel
16     // needs no inter-rank data, so the fused path is valid for ALL ranks.
```

7 参考源清单

参考源	类型	内容/作用
cpp/cuda/qcu/include/lattice_clover_multigrid.h:74-112	仓库	多 block 归约 kernel 定义
cpp/cuda/qcu/include/lattice_clover_multigrid.h:556-575	仓库	dot_coarse 多 block 路径
cpp/cuda/qcu/include/lattice_clover_multigrid.h:646-668	仓库	bistabcbg_iter_coarse 归约选择
cpp/cuda/qcu/include/lattice_clover_multigrid.h:1215-1230	仓库	fused 最粗层阈值
logs/dev76/dev76_report.md	仓库	本次工作总结
logs/dev76/AGENTS.md	仓库	套件复现指南
logs/dev76/main.py	仓库	测试套件入口

8 结论与局限

结论

三视角结论：**结构**——C++ 求解层的内核组织；**关系**——点积归约是粗层迭代的主导开销（非 dslash）；**思路**——按规模自适应选择归约/求解路径。主题结论：多 block 归约使 16x16x16x32 3L 粗层求解 10 倍加速（533→42ms），全部 3L 配置加速比提升（+5% 28%），16x16x16x32 3L 首次达到 1.0 以上。

参考背景

多重网格中粗层求解的并行归约优化是标准工程实践；本工作与 QUDA 等生产库”粗层用独立 kernel + 高效归约”的设计一致。2L 大格子 <1 为层数固有特性，非实现缺陷。

局限与风险

未验证项： 16x16x16x32 3L 加速比在 0.95-1.20 间波动（单次测量噪声大，取中值 1.08）；
CHECK_INTERVAL 延迟检查已回退但未深入验证其潜在收益（估计 <5%）； 多 block 阈值
64K 未做精细扫描（16K/32K/128K），当前值基于 196608 元素实测。